WEEK-4

Program to Implement the following Memory Allocation Methods for fixed partition a) First Fit b) Worst Fit c) Best Fit

AIM: Implementation of the following Memory Allocation Methods for fixed partition for First Fit, Worst Fit and Best Fit.

Algorithm:

1. First Fit

1. Start.
2. Read the number and sizes of memory partitions.
3. Read the number and sizes of processes.
4. For each process, search the partitions from the beginning.
5. Find the first partition whose size is greater than or equal to the process size.
6. Allocate the process to that partition.
7. Reduce the remaining partition size.
8. If no suitable partition is found, mark the process as Not Allocated.
9. Repeat for all processes.
10. Stop.

2. Worst Fit

1. Start.
2. Read the partition and process sizes.
3. For each process, search all partitions.
4. Find the largest available partition that can accommodate the process.
5. Allocate the process and reduce the partition size.
6. If no suitable partition exists, mark it as Not Allocated.
7. Repeat for all processes.
8. Stop.

3. Best Fit

1. Start.
2. Read the partition and process sizes.
3. For each process, search all partitions.
4. Find the smallest available partition that can accommodate the process.
5. Allocate the process and reduce the partition size.
6. If no suitable partition exists, mark it as Not Allocated.
7. Repeat for all processes.
8. Stop.

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program:

```
#include <stdio.h>

void firstFit(int block[], int b, int process[], int p)
{
   int allocation[20];
   int i, j;

   for(i = 0; i < p; i++)
      allocation[i] = -1;

   for(i = 0; i < p; i++)
   {
      for(j = 0; j < b; j++)
      {
         if(block[j] >= process[i])
         {
            allocation[i] = j;
            block[j] -= process[i];
            break;
         }
      }
   }

   printf("\n--- FIRST FIT ---\n");

   for(i = 0; i < p; i++)
   {
      if(allocation[i] != -1)
         printf("Process %d (%d KB) -> Partition %d\n",
               i + 1, process[i], allocation[i] + 1);
      else
         printf("Process %d (%d KB) -> Not Allocated\n",
               i + 1, process[i]);
   }
}

void worstFit(int block[], int b, int process[], int p)
{
```

```
    int allocation[20];
    int i, j, worst;

    for(i = 0; i < p; i++)
        allocation[i] = -1;

    for(i = 0; i < p; i++)
    {
        worst = -1;

        for(j = 0; j < b; j++)
        {
            if(block[j] >= process[i])
            {
                if(worst == -1 || block[j] > block[worst])
                    worst = j;
            }
        }

        if(worst != -1)
        {
            allocation[i] = worst;
            block[worst] -= process[i];
        }
    }

    printf("\n--- WORST FIT ---\n");

    for(i = 0; i < p; i++)
    {
        if(allocation[i] != -1)
            printf("Process %d (%d KB) -> Partition %d\n",
                i + 1, process[i], allocation[i] + 1);
        else
            printf("Process %d (%d KB) -> Not Allocated\n",
                i + 1, process[i]);
    }
}

void bestFit(int block[], int b, int process[], int p)
{
    int allocation[20];
    int i, j, best;
```

```
    for(i = 0; i < p; i++)
        allocation[i] = -1;

    for(i = 0; i < p; i++)
    {
        best = -1;

        for(j = 0; j < b; j++)
        {
            if(block[j] >= process[i])
            {
                if(best == -1 || block[j] < block[best])
                    best = j;
            }
        }

        if(best != -1)
        {
            allocation[i] = best;
            block[best] -= process[i];
        }
    }

    printf("\n--- BEST FIT ---\n");

    for(i = 0; i < p; i++)
    {
        if(allocation[i] != -1)
            printf("Process %d (%d KB) -> Partition %d\n",
                i + 1, process[i], allocation[i] + 1);
        else
            printf("Process %d (%d KB) -> Not Allocated\n",
                i + 1, process[i]);
    }
}

int main()
{
    int block[20], process[20];
```

```
    int block1[20], block2[20], block3[20];
    int b, p, i;

    printf("=========================================\n");
    printf("       FIXED PARTITION MEMORY ALLOCATION\n");
    printf("=========================================\n");

    printf("Enter number of partitions: ");
    scanf("%d", &b);

    printf("Enter partition sizes:\n");
    for(i = 0; i < b; i++)
    {
       printf("Partition %d: ", i + 1);
       scanf("%d", &block[i]);
    }

    printf("\nEnter number of processes: ");
    scanf("%d", &p);

    printf("Enter process sizes:\n");
    for(i = 0; i < p; i++)
    {
       printf("Process %d: ", i + 1);
       scanf("%d", &process[i]);
    }

    /* Make separate copies for each method */
    for(i = 0; i < b; i++)
    {
       block1[i] = block[i];
       block2[i] = block[i];
       block3[i] = block[i];
    }

    firstFit(block1, b, process, p);
    worstFit(block2, b, process, p);
    bestFit(block3, b, process, p);

    return 0;
}
```

VNR VJIET

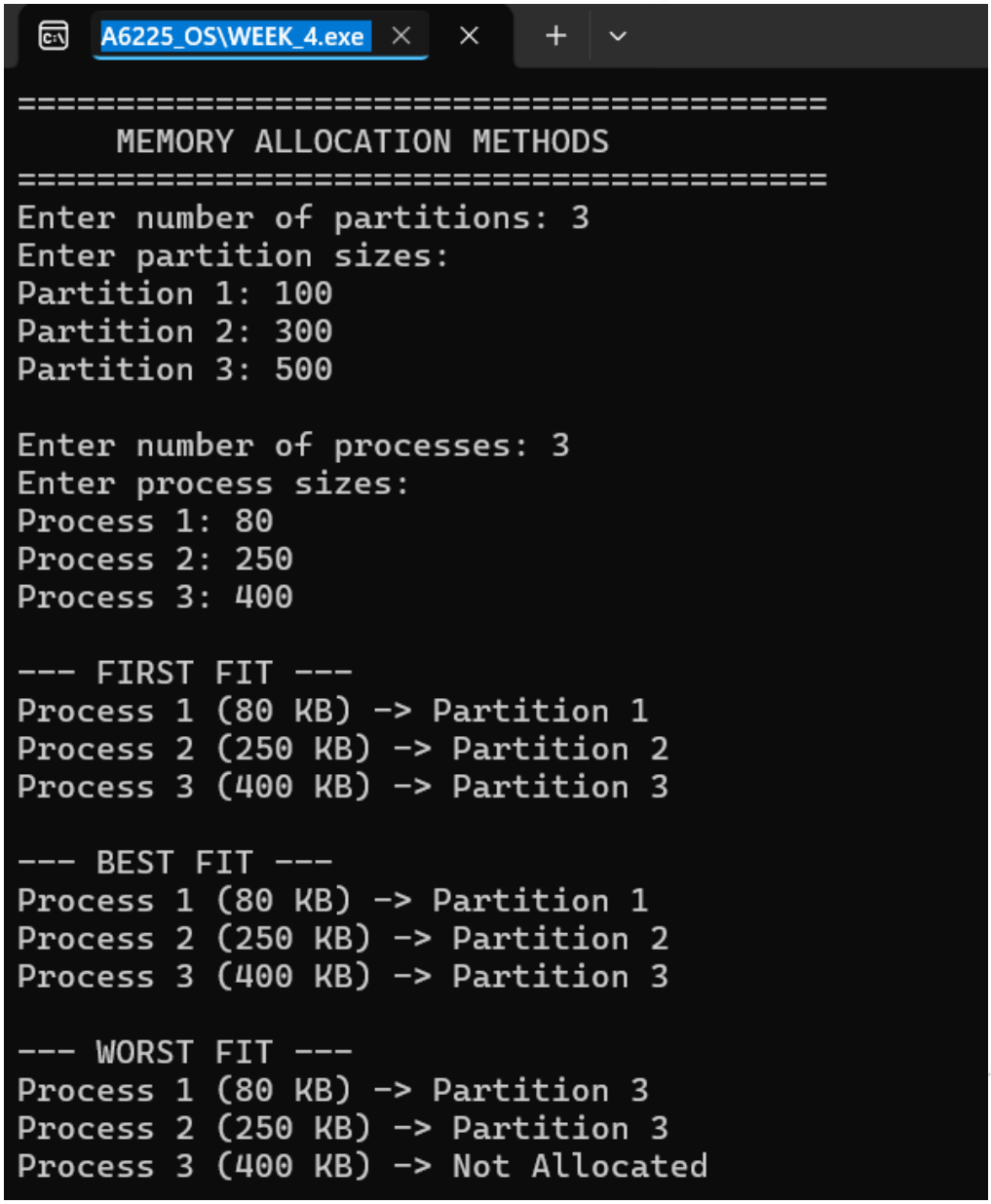
Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

OUTPUT:

TESTCASE 1:



TESTCASE 2:

```
A6225_OS\WEEK_4.exe
Partition 4: 400

Enter number of processes: 4
Enter process sizes:
Process 1: 300
Process 2: 200
Process 3: 500
Process 4: 100

--- FIRST FIT ---
Process 1 (300 KB) -> Partition 2
Process 2 (200 KB) -> Partition 3
Process 3 (500 KB) -> Not Allocated
Process 4 (100 KB) -> Partition 1

--- BEST FIT ---
Process 1 (300 KB) -> Partition 2
Process 2 (200 KB) -> Partition 4
Process 3 (500 KB) -> Partition 3
Process 4 (100 KB) -> Partition 3

--- WORST FIT ---
Process 1 (300 KB) -> Partition 3
Process 2 (200 KB) -> Partition 4
Process 3 (500 KB) -> Not Allocated
Process 4 (100 KB) -> Partition 2
```

TESTCASE 3:

```
A6225_OS\WEEK_4.exe
========================================
      MEMORY ALLOCATION METHODS
========================================
Enter number of partitions: 5
Enter partition sizes:
Partition 1: 100
Partition 2: 500
Partition 3: 200
Partition 4: 300
Partition 5: 600

Enter number of processes: 4
Enter process sizes:
Process 1: 212
Process 2: 417
Process 3: 112
Process 4: 426

--- FIRST FIT ---
Process 1 (212 KB) -> Partition 2
Process 2 (417 KB) -> Partition 5
Process 3 (112 KB) -> Partition 2
Process 4 (426 KB) -> Not Allocated

--- BEST FIT ---
Process 1 (212 KB) -> Partition 4
Process 2 (417 KB) -> Partition 2
Process 3 (112 KB) -> Partition 3
Process 4 (426 KB) -> Partition 5

--- WORST FIT ---
Process 1 (212 KB) -> Partition 5
Process 2 (417 KB) -> Partition 2
Process 3 (112 KB) -> Partition 5
Process 4 (426 KB) -> Not Allocated
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

WEEK-5

PROGRAM TO IMPLEMENT PAGING TECHNIQUE OF MEMORY MANAGEMENT

AIM: Implementation of Paging Technique of Memory Management

ALOGORITHM:

- Start
- Read the number of pages.
- Read the page size.
- Read the page table, which contains the frame number for each page.
- Read the logical address.
- Calculate the page number:
- Page Number $= \frac{\text{Logical Address}}{\text{Page Size}}$

- Calculate the offset:
- Offset = Logical Address%Page Size

- Use the page number to find the corresponding frame number from the page table.
- Calculate the physical address:
- Physical Address = (Frame Number × Page Size) + Offset

- Display the page number, offset, frame number, and physical address.
- Stop

PROGRAM:

```
#include <stdio.h>

int main()
{
   int pages, pageSize;
   int pageTable[20];
   int logicalAddress;
   int pageNumber, offset;
   int frameNumber, physicalAddress;
   int i;

   printf("=======================================\n");
   printf("       PAGING MEMORY MANAGEMENT\n");
   printf("=======================================\n");

   printf("Enter number of pages: ");
   scanf("%d", &pages);

   printf("Enter page size: ");
   scanf("%d", &pageSize);

   printf("\nEnter Page Table:\n");

   for(i = 0; i < pages; i++)
   {
      printf("Page %d -> Frame: ", i);
      scanf("%d", &pageTable[i]);
   }
```

```
printf("\nEnter Logical Address: ");
 scanf("%d", &logicalAddress);

 /* Calculate page number and offset */
 pageNumber = logicalAddress / pageSize;
 offset = logicalAddress % pageSize;

 /* Check whether page number is valid */
 if(pageNumber >= pages)
 {
    printf("\nInvalid Logical Address!\n");
 }
 else
 {
    /* Find frame number from page table */
    frameNumber = pageTable[pageNumber];

    /* Calculate physical address */
    physicalAddress = (frameNumber * pageSize) + offset;

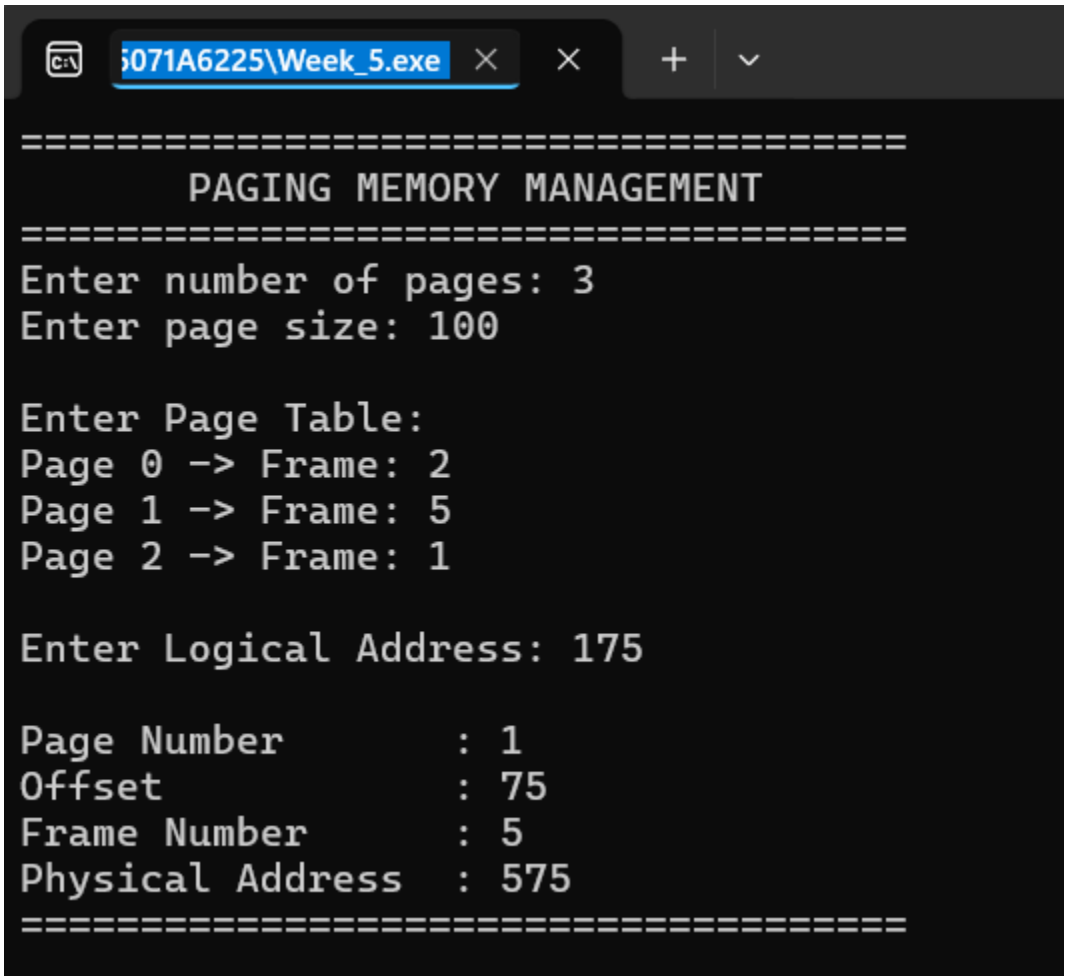
    printf("\nPage Number      : %d", pageNumber);
    printf("\nOffset           : %d", offset);
    printf("\nFrame Number     : %d", frameNumber);
    printf("\nPhysical Address : %d\n", physicalAddress);
 }
```

```
    printf("=====================================\n");

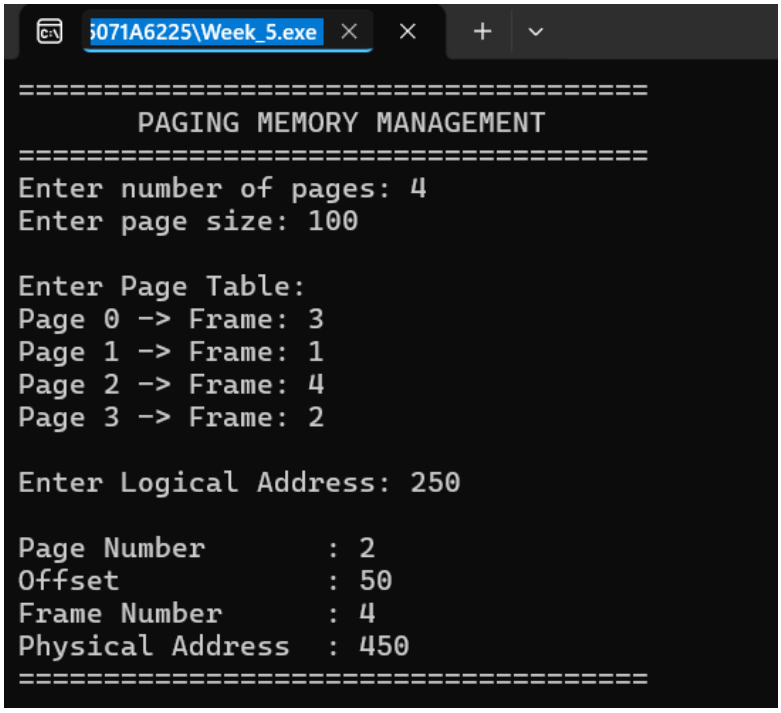

    return 0;
}
```

OUTPUT:

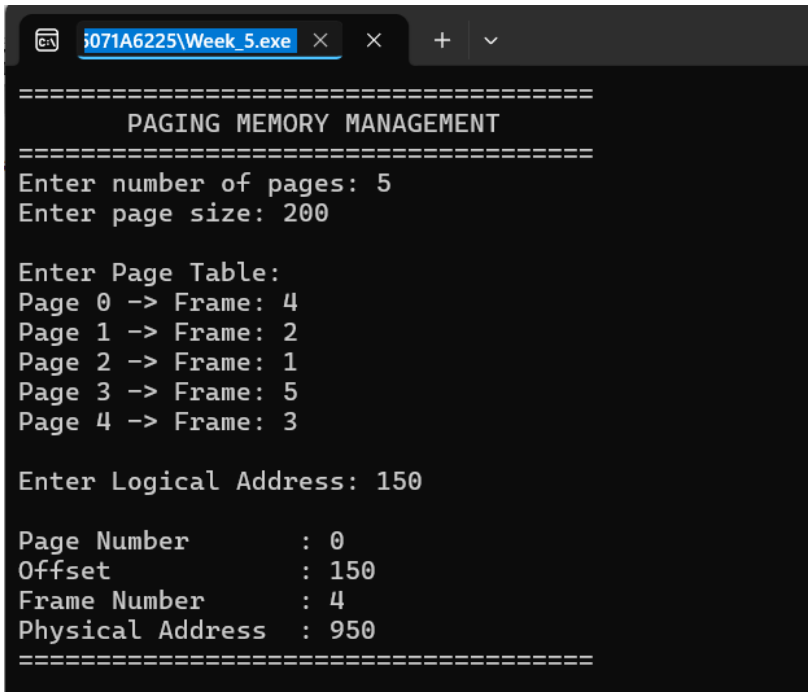
TEST CASE 1:



TEST CASE 2:


5071A6225\Week_5.exe
PAGING MEMORY MANAGEMENT
Enter number of pages: 4
Enter page size: 100
Enter Page Table:
Page 0 -> Frame: 3
Page 1 -> Frame: 1
Page 2 -> Frame: 4
Page 3 -> Frame: 2
Enter Logical Address: 250
Page Number : 2
Offset : 50
Frame Number : 4
Physical Address : 450

TEST CASE 3:


5071A6225\Week_5.exe
PAGING MEMORY MANAGEMENT
Enter number of pages: 5
Enter page size: 200
Enter Page Table:
Page 0 -> Frame: 4
Page 1 -> Frame: 2
Page 2 -> Frame: 1
Page 3 -> Frame: 5
Page 4 -> Frame: 3
Enter Logical Address: 150
Page Number : 0
Offset : 150
Frame Number : 4
Physical Address : 950